THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL ENGINEERING / GROUP EE14

INTERNAL TECHNICAL REVIEW / NOVEMBER 2026

VIDYUT

FAULT CLASSIFICATION / PROTECT-90 / 48 CHANNELS AT 6.4 kHz

Intelligent fault detection and classification in high-voltage transmission networks

Technical report

DOCUMENT 001 / REPORT

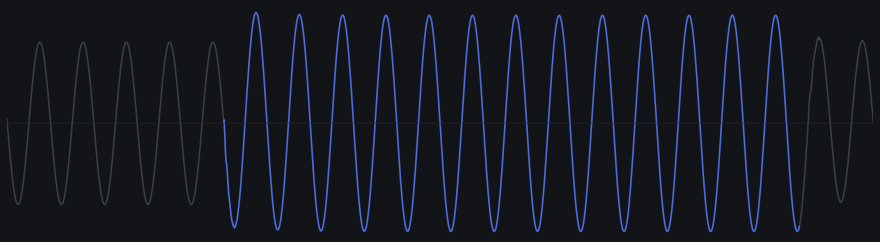
Ishita Sharma 1025040017
Prathamesh Narang 1025040101
Prakhar Goel 1025040103

AI For Engineers
Submitted to Dr. Krishna Kumar Gupta
Department of Electrical Engineering
Thapar Institute of Engineering and Technology
Academic session 2026-27

CLASS REGISTER

0 LLL 1 LL 2 SLG 3 LLG

SPECIMEN 0000 / PHASE L1 CURRENT / 0.33-0.73 s / ONE EVENT OF 9,022



9,022 EPISODES / 90 kV DOUBLE CIRCUIT / 48 CHANNELS / 6,400 SAMPLES

Contents

Abstract 2
1 Introduction 2
2 Problem formulation 3
3 Literature review and background 4
4 Dataset: PROTECT-90 5
5 Data preprocessing 7
6 Proposed methodology and implementation 9
7 Results 11
8 Discussion 14
9 Applications, limitations and future scope 14
10 Conclusion 16
References 16
Appendix A Reproduction 17

Figures and tables are numbered in order of appearance; every metric in this report is traceable to a named artefact in the project repository. All results describe PROTECT-90, a simulated benchmark; no field measurement was used.

Companion deliverables: a ten-slide deck (VIDYUT_Research_Presentation.pptx) with speaker notes, an evidence and consistency audit, a reproducibility checklist and an IEEE reference list.

SECTION VIDYUT / EE14

Abstract

Transmission-line faults must be identified from the voltages and currents they leave behind, and the four standard categories, three-phase, line-to-line, single line-to-ground and double line-to-ground, remain the first decision any protection study has to make. This project asks whether a learned classifier can make that decision from short windows of synchronised waveforms, and how far the resulting accuracy can be trusted.

The work uses PROTECT-90, an open benchmark of 9,022 electromagnetic transient simulations on a 90 kV double-circuit network with 48 channels at eight locations, sampled at 6.4 kHz. Episodes were split 70/15/15 at episode level before any windowing, and 256-sample windows were drawn from inside the fault interval, giving 105,728 training, 22,908 validation and 22,519 test windows. Three pipelines were trained: a random forest on 192 engineered descriptors, a compact one-dimensional CNN with 37,316 parameters, and a hybrid that concatenates the CNN embedding with the descriptors and classifies with a forest.

On validation the CNN reached 99.92% accuracy and 0.9992 macro-F1 against 96.11% for the forest, and was selected on macro-F1 alone. On the test split it reached 99.90% window accuracy with 22 errors in 22,519 windows, and 99.63% macro-F1 over 1,354 episodes when its windows were aggregated by majority vote.

The result is a property of the benchmark, not of a protection function. Windows are drawn from inside the fault, the data are simulated, one seed and one split were used, and the accuracy therefore describes a well-posed classification problem on separable synthetic signals. Establishing anything about field behaviour would require the robustness and recording experiments that this project identifies but does not perform.

SECTION 1 VIDYUT / EE14

Introduction

1.1 Transmission networks and the cost of a fault

A high-voltage transmission network is a machine for moving bulk power between a small number of nodes. Its lines are exposed, its fault currents are large, and the interval between fault inception and isolation is measured in cycles. Protection engineering has therefore always treated fault identification as a real-time decision problem: which phases are involved, whether earth is involved, where the fault is, and whether the disturbance is a fault at all.

Conventional protection answers the first of these questions with measurement logic. Distance elements compare impedance, overcurrent elements compare magnitude against a setting, and sequence-component elements exploit the symmetry that a fault breaks. These methods are fast, deterministic and auditable, and they remain the basis of every deployed scheme. Their weakness is that the settings encode assumptions about topology, source strength and load, and that a changing network

can invalidate those assumptions without any component failing.

This is the gap that learning-based classification addresses. A classifier is not given a threshold; it is given examples of what each fault category looks like in the measured waveforms, and it learns a decision boundary from them. Whether that boundary generalises is an empirical question, and it is the question this project is designed to answer honestly rather than optimistically.

1.2 The four categories

A short circuit on a three-phase line can be classified by which conductors touch. Single line-to-ground (SLG) faults involve one phase and earth and are the most common in service. Line-to-line (LL) faults involve two phases. Double line-to-ground (LLG) faults involve two phases and earth. Three-phase (LLL) faults involve all three and are the most severe and the least common. The dataset labels these as integers 0 to 3 in the order LLL, LL, SLG, LLG, and this project keeps that encoding unchanged.

The four categories are not equally easy. An LLL fault leaves a symmetrical current pattern distorted in magnitude; an SLG fault destroys the symmetry entirely and produces a large zero-sequence component. LL and LLG differ only in whether the fault path reaches earth, and that difference can be small in a short window. A classifier that separates all four is therefore doing more than detecting that something happened.

SECTION 2 VIDYUT / EE14

Problem formulation

2.1 Problem statement

Let one window of measured data be a matrix of 48 channels by 256 samples, sampled at 6.4 kHz, so that the window spans 40 ms or two nominal 50 Hz cycles. The task is to learn a mapping

$$\hat{y} = f_\theta(x), \; f_\theta : [48 \times 256] \rightarrow \{0, 1, 2, 3\} \tag{1}$$

where x is the window, θ are learned parameters, and $\hat{y}$ is the predicted fault category. The window inherits the label of the episode from which it was cut, so every window of one simulation carries the same class. Two consequences follow immediately and shape the whole project. First, windows from one episode are not independent observations, so the number of windows is not the number of independent trials. Second, splitting must be done at the level of the episode, because windows cut from the same simulation share its parameters and would leak into any split made at window level.

2.2 What this project does not attempt

The scope is classification only. No fault location, no distance estimate, no fault-resistance estimate, no detection latency, no trip decision and no hardware implementation are attempted. The classifier is an offline analysis of recorded windows, and every statement in this report is bounded by that.

2.3 Objectives

1. Reconstruct the PROTECT-90 pipeline: verify the archive, the metadata and the class encoding, and establish the true dimensions and split.
2. Build an episode-level split before windowing, with no episode spanning two splits.

3 Implement three pipelines of increasing representational power on identical windows.

4 Select one model on validation performance only, and evaluate it once on the test split.

5 Report accuracy at both window and episode level, with the unit stated for every number.

6 State the limits of the result, including the failure that preceded it.

SECTION 3 VIDYUT / EE14

Literature review and background

The literature on transmission-line fault classification divides into three traditions: measurement logic, signal processing with a shallow classifier, and end-to-end learning. Each contributes something the present work uses, and each carries an assumption worth naming.

3.1 Conventional and signal-processing methods

Classical schemes derive features that correspond to physical quantities: impedance, symmetrical components, travelling-wave arrival times. Jamil, Sharma and Singh [6] train a feed-forward network on time-domain features extracted from simulated transmission-line faults and report classification of the standard categories, which establishes the pattern this project follows: engineer a small descriptor, then classify. The attraction is interpretability, and the cost is that the descriptor fixes what the model can see.

The signal-processing tradition replaces hand-picked descriptors with transforms. Wavelet and Hilbert-Huang decompositions localise the transient in time and frequency, and a classifier is then applied to the coefficients. These methods are strong on non-stationary transients, but they add a transform stage whose parameters must themselves be chosen, and their performance is usually reported on a single simulated topology.

3.2 One-dimensional convolutional networks

A one-dimensional CNN learns its own filters directly from the waveform, which removes the descriptor bottleneck. Kiranyaz et al. [4] survey the family and argue that 1-D convolution is the natural inductive bias for sensor signals: local receptive fields, weight sharing along time, and a hierarchy that moves from waveform shape to fault signature. The same survey notes the recurring weakness, that reported accuracy is often a property of a synthetic dataset with highly separable classes.

Turanli and Benteşen Yakut [5] apply a 1-D CNN to fault classification across real, synthetic and public datasets and report near-perfect scores on generated data alongside visibly lower scores on recorded data. That contrast is the most useful result in the recent literature for this project, because it predicts exactly the pattern observed here.

3.3 Temporal convolution and hybrid models

Tunio et al. [7] use a temporal convolutional network with systematic feature extraction on a 500 kV network and report 99.9% classification accuracy under randomised fault conditions. The number is comparable to the one obtained here, on a similarly simulated dataset, which is a reminder that such figures describe separability rather than deployment readiness.

3.4 Ensembles and tabular learners

Random forests [3] remain a strong baseline for small, tabular descriptor sets. They are fast to fit, resistant to irrelevant features, and produce a decision that can be inspected tree by tree. Their limitation in this setting is structural: once a 40 ms waveform has been reduced to four statistics per channel, the temporal shape that distinguishes LL from LLG has already been discarded.

3.5 Benchmarks and the reporting problem

The PROTECT-90 dataset and its accompanying paper [1], [2] exist precisely because the field lacked a common benchmark. The dataset documentation asks users to report task definition, input observability, window length, preprocessing, split strategy and evaluation unit, and to say whether results are window-based or episode-based. This report follows that checklist explicitly, and the distinction between the two units is treated as a result in its own right.

SECTION 4 VIDYUT / EE14

Dataset: PROTECT-90

4.1 Provenance

PROTECT-90 version 1.0.0 was released on 1 July 2026 under CC BY 4.0, with DOI 10.5281/zenodo.21109169 [1]. It was produced within the DFG project Netzschutz-KI at Friedrich-Alexander-Universität Erlangen-Nürnberg and is described in an accompanying paper [2]. The episodes are electromagnetic transient simulations of a fixed 90 kV double-circuit topology; the dataset documentation states that the released values are secondary-side voltage and current quantities, consistent with digital fault recording practice.

The record contains a metadata table, hv_double_line_90kv_labels.csv, with one row per episode, and an archive of waveform files, one pickled pandas DataFrame per episode. Each frame has 6,400 rows and 49 columns: 48 waveform channels plus a time column, time_s. No feature engineering is applied by the publisher, and no train/test split is provided; users are asked to define one.

Table 1. Dataset characteristics as verified from the released metadata and documentation.

Property	Value
Episodes	9,022
Network	90 kV double circuit, buses 1-2-3
Measurement locations	8
Channels	48 (3 voltages and 3 currents per location)
Sampling frequency	6.4 kHz, 128 samples per 50 Hz cycle
Episode length	1.0 s, 6,400 samples
Fault classes	4 (LLL, LL, SLG, LLG)
Metadata	sample_id, sc_type, fault_target, start and end times, location, resistance
Licence and DOI	CC BY 4.0, 10.5281/zenodo.21109169

Domain randomisation covers line parameters, load operating points, external grid short-circuit strength, voltage magnitude and phase angle, parallel-line switching states, fault type, fault resistance, fault location and fault inception time. All scenarios pass load-flow and numerical stability checks before release. The

dataset is therefore diverse in operating condition while remaining fixed in topology, which matters when reading the results: the model learns this network, not transmission networks in general.

4.2 Classes and encoding

The label column is sc_type, and its mapping was verified against the companion repository documentation rather than assumed. The order is not alphabetical and not intuitive, which is why it is stated here once and used consistently in every table and figure.

Table 2. Fault categories, encoded labels and episode counts.

sc_type	Class	Description	Episodes	Share
0	LLL	three phases joined	2,273	25.19 %
1	LL	two phases joined	2,251	24.95 %
2	SLG	one phase to earth	2,279	25.26 %
3	LLG	two phases to earth	2,219	24.60 %

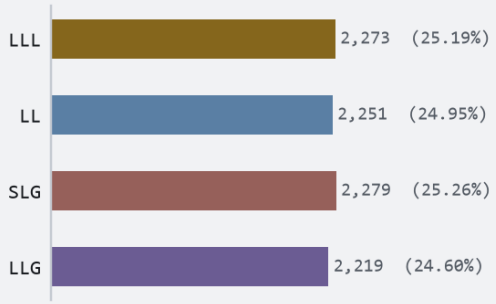


Figure 1. Episode count per class. The four classes are balanced to within 0.66 percentage points, so accuracy is a fair summary and no reweighting is needed for class imbalance. Derived from the metadata table.

The balance is a property of the generator rather than of the power system, and it removes a difficulty that field data would not remove. An SLG fault is roughly five to ten times more common than an LLL fault in service; here the classes are almost equal, so the classifier is never asked to trade recall between a common and a rare class.

4.3 Fault interval statistics

Each episode carries t_evnt_start and t_evnt_end. Across the dataset the fault lasts 226.3 ms on average (median 218.3 ms), with a minimum of 0.12 ms and a maximum of 594.1 ms. The distribution matters because the window is 40 ms long: 828 episodes (9.2%) hold a fault shorter than one window, and 404 are shorter than a single 20 ms cycle. These episodes are not discarded; their windows are clamped inside the episode and carry lower fault coverage. Any accuracy figure therefore mixes easy long faults with short ones, in the proportions the generator produced.

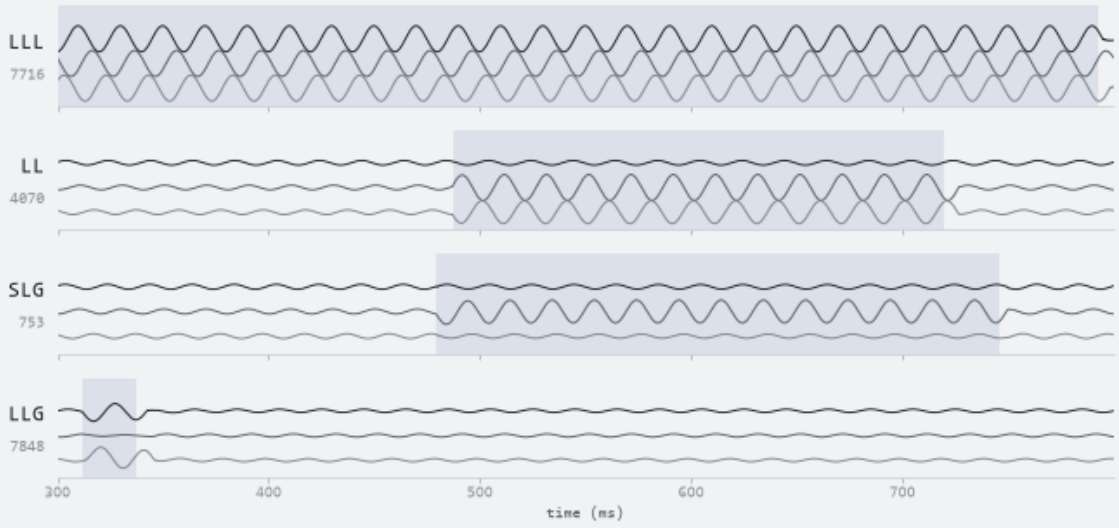


Figure 2. One episode from each class, three phase currents, with the fault interval shaded. Sample identifiers are those of the released files. The traces are z-scored with the training statistics, so amplitudes are comparable across panels; the point of the figure is the change in shape, not the absolute magnitude.

SECTION 5 VIDYUT / EE14

Data preprocessing

5.1 Integrity and validation

The archive was checked against the published size and MD5 checksum, and the extracted file count matches the metadata row count of 9,022. Every waveform was parsed and checked for shape, for missing and infinite values, and for a monotonic time axis. Metadata was checked for duplicate identifiers, invalid class codes, invalid timing and unknown fault targets; none were found. One episode, sample 7154, fails to unpickle and is excluded. It is the only exclusion, and it is recorded in the window shape integrity report rather than silently dropped.

Archive integrity and parsing integrity are distinct claims, and it is worth being precise about which is being made. The checksum establishes that the downloaded archive is the published archive. Successful parsing of all 9,022 frames establishes that each file is readable and dimensionally correct. Neither establishes that the simulation inside is physically right, which is the publisher's claim to defend, not this project's.

5.2 Episode-level split

The split is made on episode identifiers before any window exists. A two-stage stratified procedure holds out 30% of episodes, then halves that remainder, giving 70/15/15 on sc_type with seed 42. The result is 6,315 training, 1,353 validation and 1,354 test episodes, which sum to 9,022.

The reason for splitting at episode level is leakage. Windows are cut with 75% overlap, and every window of an episode inherits that episode's random line parameters, load point, fault resistance and inception time. A window-level split would place near-duplicate windows on both sides of the boundary and inflate validation accuracy by an amount that cannot be estimated afterwards. The verification script confirms zero episodes appear in more than one split.

Table 3. Split sizes at both units. Episode counts are the split; window counts are what those episodes produce after windowing.

Split	Episodes	Windows	Share of windows
Training	6,315	105,728	69.95 %
Validation	1,353	22,908	15.16 %
Test	1,354	22,519	14.90 %

5.3 Window extraction

Windows are 256 samples long, 40 ms, cut with a stride of 64 samples and capped at 24 windows per episode. The cap keeps one long fault from dominating the training set, at the price of sampling long faults less densely than short ones. Windows are drawn from the fault interval: an onset-centred window is taken first, then the stride sweep fills the remaining budget inside the fault span, clamped to the episode boundary. Mean fault coverage is 0.964, and every window overlaps the fault at least partially.

This placement is the correction of a defect, and the defect is part of the record. An earlier version of the sampler swept from sample zero. Because the fault begins near t = 0.43 s of a 1 s record and the 24-window budget is filled from the start, only 7.0% of windows on a 200-episode sample overlapped the fault, with mean coverage 0.035. All three models trained on those windows scored at chance: macro-F1 of 0.281 for the forest, 0.233 for the CNN and 0.263 for the hybrid, against 0.25 expected from four equal classes. Those archived results are retained in a separate directory rather than overwritten.

The fix is small and the lesson is not. The sampler had been given a frame with its time column already removed, inferred a missing sample interval, and silently fallen through to the plain sweep; its fault-alignment branch never executed. The loader now raises rather than falling back, and the coverage metric is computed from the returned window starts against a genuine fault interval, so the failure cannot recur silently. A second-order version of the same mistake is also on record: the first attempt at a fix measured coverage against the broken bounds and reported 1.0000 while still producing pre-fault windows.

5.4 Normalisation and descriptors

Each channel is standardised with the training mean and standard deviation,

$$z = (x - \mu) / \sigma \text{ with } \mu, \sigma \text{ taken from the training windows, per channel} \quad (2)$$

fitted once on the training windows and applied unchanged to validation and test. Fitting on the full dataset would leak a small amount of distributional information; fitting per split would make the splits incomparable. For the descriptor pipeline the stored z-score is inverted with the same training statistics before the statistics are computed, so those features are in amperes and volts rather than in standard deviations.

Four descriptors are computed per channel over the 256 samples: RMS, standard deviation, peak-to-peak amplitude and maximum absolute amplitude. With 48 channels this gives 192 features per window. Two of the four are nearly redundant for a sinusoid (RMS and standard deviation differ by $\sqrt{2}$), which is an inefficiency retained for simplicity rather than a design choice that was measured.

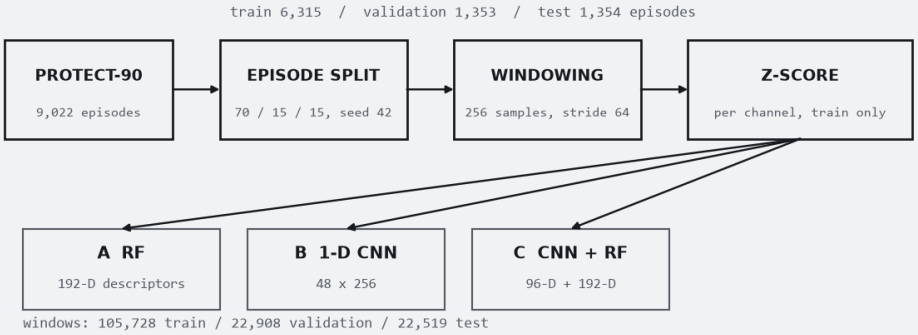


Figure 3. The implemented pipeline. Episodes are split before windowing, windows are cut inside the fault interval, and the three models see the same normalised arrays with different input representations.

SECTION 6 VIDYUT / EE14

Proposed methodology and implementation

6.1 Random forest on engineered descriptors

The first pipeline is a random forest [3] of 400 trees with a minimum of 2 samples per leaf and balanced class weights, fitted on the 192 descriptors. Each tree is grown on a bootstrap sample with a random subset of features considered at every split, and the forest votes across trees. The model is deliberately conventional: it establishes what the descriptors alone can do, and it is the baseline against which the temporal model must justify itself.

6.2 One-dimensional CNN on the raw window

The second pipeline takes the 48 × 256 window directly. A one-dimensional convolution applies a learned kernel along the time axis:

$$y_c[n] = \varphi(b_c + \Sigma_k \Sigma_{ch} w_{c,ch}[k] \cdot x_{ch}[n + k]) \quad (3)$$

where c indexes output channels, k indexes the kernel, ch indexes input channels and φ is a pointwise non-linearity. Because the same kernel is applied at every time step, the layer detects a waveform motif wherever it occurs, which is exactly the invariance a fault signature needs.

The implemented architecture is three convolutional blocks, 48 → 32 with kernel 5, 32 → 64 with kernel 5, and 64 → 96 with kernel 3, each followed by batch normalisation and a rectified linear unit, with max pooling after the first two blocks. A global average pool collapses the time axis to a 96-dimensional embedding, and a single linear layer maps it to four logits. The model has 37,316 parameters, which is small enough to train on a CPU in minutes and small enough that the result cannot be attributed to capacity.

6.3 Hybrid: embedding plus descriptors

The third pipeline concatenates the 96-dimensional CNN embedding with the 192 descriptors and fits a random forest of 400 trees on the resulting 288-dimensional vector. The intention is to give the forest the temporal representation the descriptors lack. The embedding is extracted once from the trained encoder,

so the hybrid does not retrain the convolutional part.

6.4 Training protocol

All three models train on the same fixed subset of 16,384 windows, drawn as a class-balanced, stratified sample of the 105,728 available training windows with seed 42. The subset covers 5,328 of 6,314 windowed training episodes (84.4%) with a median of 3 windows per episode. Validation and test use every window of their splits.

The CNN is trained for 20 epochs with AdamW at a learning rate of 1e-3 and weight decay 1e-4, batch size 128, cross-entropy loss on CPU. Early stopping with patience 5 was configured and never triggered; the best validation-loss checkpoint, epoch 18, is the one used. No hyperparameter search was run. Both forest settings and CNN settings were fixed a priori, which is a limitation on how much can be claimed for the architecture and a deliberate saving on compute.

Table 4. Implementation and training configuration.

Item	Value
Language and runtime	Python 3.14, CPU only (16 cores, 16 GB RAM)
Libraries	NumPy 2.3.5, pandas 3.1.0, scikit-learn 1.9.1, PyTorch 2.9.1, Matplotlib 3.11.2, joblib 1.6.0
Input tensor	48 channels × 256 samples, float32
Training subset	16,384 windows, seed 42
CNN optimizer	AdamW, lr 1e-3, weight decay 1e-4
Batch size and epochs	128, 20 epochs, no early stop
CNN parameters	37,316
Random forest	400 trees, leaf ≥ 2, balanced weights
Artifacts	expA_rf_features_model.pkl, expB_cnn_best.pt, expC_hybrid_rf_model.pkl



Figure 4. CNN training and validation loss over 20 epochs, logarithmic axis. The best validation loss occurs at epoch 18; the rise at epoch 20 is why the checkpoint rather than the final weights are used.

6.5 Model selection

Selection used validation macro-F1 only, computed once per model on the same 22,908 windows. The test split took no part in the choice. This matters because it is easy to describe a test set as untouched while having looked at it during diagnosis; that is what happened here, when the legacy defective-window test scores were inspected while the sampler fault was being found. The precise claim is therefore narrow and

defensible: the test split did not choose the model, and the selected model was scored on it once.

SECTION 7 VIDYUT / EE14

Results

7.1 Validation

On the validation split the ranking is unambiguous. The forest on descriptors reaches 96.11% accuracy; the CNN reaches 99.92%; the hybrid sits between them at 99.89%. In error counts, out of 22,908 windows, that is 891, 19 and 25 mistakes respectively.

Table 5. Validation performance of the three pipelines on 22,908 windows.

Model	Input	Accuracy	Macro precision	Macro recall	Macro F1	Errors
A RF	192 descriptors	96.11	0.9608	0.9608	0.9608	891
B 1-D CNN	48 × 256	99.92	0.9992	0.9992	0.9992	19
C hybrid	288 (96 + 192)	99.89	0.9989	0.9989	0.9989	25

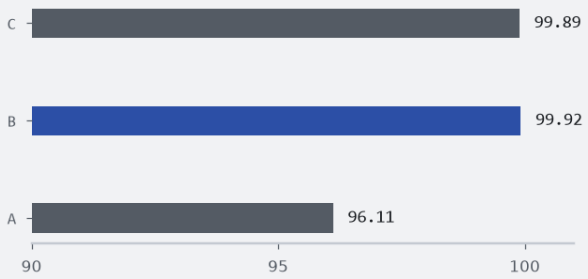


Figure 5. Validation accuracy by model. Panel B is the selected model; the gap to A is 3.8 percentage points and the gap to C is 0.026 points.

The gap between A and B is the finding worth explaining. The two models see the same windows; the difference is representation. Four statistics per channel retain magnitude and discard shape, so LL and LLG become hard to separate. In A's validation confusion matrix, 739 of its 891 errors lie between those two classes, which is 82.9% of all its mistakes. The CNN does not have that failure mode, because a kernel that responds to the shape of the fault current is still available to it after pooling.

The hybrid result is more interesting than a simple ranking suggests. Adding 192 descriptors to the 96-dimensional embedding costs 0.026 percentage points relative to the CNN alone, six extra errors out of 22,908, and no significance test was run on a difference of that size. The honest reading is that the two models are indistinguishable on this validation split, and that the descriptors add nothing once the encoder has seen the waveform. Whether they would help under noise or with fewer channels is untested.

7.2 Test performance at window level

The selected model was evaluated once on the test split, 22,519 windows from 1,354 episodes it had never seen. It classified 22,497 correctly, giving an accuracy of 99.90%, macro precision 0.9990, macro recall 0.9990 and macro F1 0.9990. Mean confidence on the test windows was 0.9988.

Table 6. Test performance of the selected model, window level, n = 22,519.

Metric	Value
Accuracy	0.9990 (99.90 %)
Macro precision	0.9990
Macro recall	0.9990
Macro F1	0.9990
Weighted F1	0.9990
Incorrect predictions	22 of 22,519
Mean confidence	0.9988

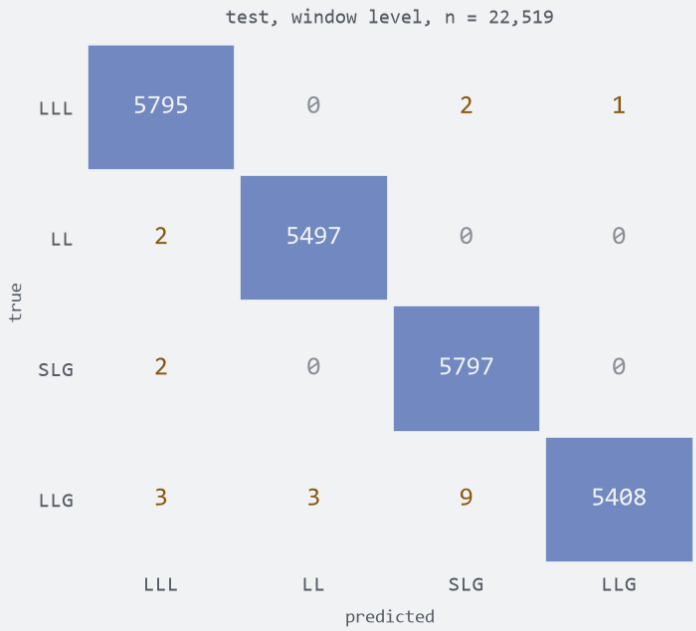


Figure 6. Test confusion matrix, window level. Rows are true classes and columns predicted; 22 of 22,519 windows fall off the diagonal. Amber numerals mark errors, cobalt blocks the diagonal.

The confusion matrix is sparse in a way that repays reading. Fifteen of the 22 errors start at LLG, and nine of those land on SLG. Only seven errors in the entire test set separate a grounded fault from an ungrounded one, which is the distinction a distance element finds hardest. The model is therefore not failing at the general problem; it fails at the boundary between two grounded classes that differ in how many phases carry fault current.

Table 7. Per-class test performance, window level.

Class	Support	Precision	Recall	F1
LLL	5,798	0.9988	0.9995	0.9991
LL	5,499	0.9995	0.9996	0.9995
SLG	5,799	0.9981	0.9997	0.9989
LLG	5,423	0.9998	0.9972	0.9985

7.3 Episode-level performance and the unit problem

A window-level rate is not the rate at which a record is classified correctly. Because 2 to 24 windows come from the same simulation, the effective sample size is closer to 1,354 than to 22,519, and a per-window

metric is optimistic relative to the question a user would actually ask. To answer that question the window predictions were mapped back to their episodes and aggregated by majority vote, with ties broken by mean probability and then by lowest class index. No ties occurred.

The mapping was reconstructed rather than read, because the stored arrays carry no episode identifier. The reconstruction re-runs the windowing function on each test episode, z-scores it with the saved statistics, and compares the result with the stored block at the hypothesised offset; the maximum absolute difference is 3.8e-06, consistent with float32 rounding, and recomputing the window-level metrics from the loaded checkpoint reproduces the stored confusion matrix exactly. Averaging probabilities instead of counting votes produces identical predictions on all 1,354 episodes.

Table 8. Episode-level test performance, majority vote over windows, n = 1,354.

Metric	Value
Episodes evaluated	1,354
Episodes correct	1,349
Episodes wrong	5
Episode accuracy	0.9963 (99.63 %)
Macro F1	0.9963
Windows per episode	min 2, median 20, max 24
Ties encountered	0

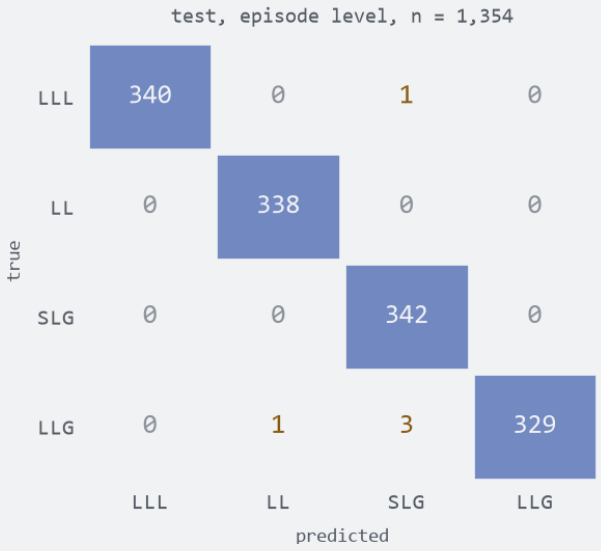


Figure 7. Episode-level confusion matrix. One LLL episode is called SLG, one LLG episode is called LL, and three LLG episodes are called SLG; the remaining 1,349 are correct.

The two units differ by 0.27 percentage points, 99.90% against 99.63%, which is smaller than the difference between an optimistic and a careless claim, and larger than zero. Part of the gap is arithmetic: a single wrong window out of 20 does not change a majority vote, so episode accuracy is more forgiving than window accuracy when windows are correlated. The two numbers answer different questions and are never pooled in this report.

7.4 What the numbers establish

Three statements are supported. The four classes are separable from 40 ms of 48-channel data in this benchmark, to within 0.1 percentage points of perfect at window level. A learned temporal representation is worth 3.8 points of accuracy over four statistics per channel, and that gain is concentrated where the

descriptor is blind, between LL and LLG. And the model's errors are not random: they cluster in the LLG boundary, which is a specific, investigable weakness rather than a general failure.

SECTION 8 VIDYUT / EE14

Discussion

The most important thing to say about these results is that they are a property of the benchmark. PROTECT-90 was generated with randomised line parameters, loads and fault conditions, which makes it a good benchmark and a hard test of generalisation across operating points. But the fault type determines the phase-current pattern directly, the windows are drawn from inside the fault, the network topology is fixed, and the measurements are noise-free simulation outputs. A model with 37,316 parameters reaching 99.9% under those conditions is reporting separability, and separability is exactly what the generator built in.

This is why the A-versus-B comparison is more informative than either absolute number. The forest and the CNN see identical windows and differ only in representation, so a 3.8-point gap measures the value of temporal structure rather than the difficulty of the task. Conversely the near-tie between B and C says that the descriptors add nothing the encoder has not already captured, which is the expected outcome if the encoder is adequate and worth knowing if it is not.

The training subset deserves scrutiny. All three models saw 16,384 windows, 15.5% of the available training data. The CNN saturates well before that limit, but the two forests plausibly would not, and the honest position is that A and C are reported at a disadvantage that was not measured. The reason for the subset was compute: the full pipeline, including featurisation and forest fitting, runs in about four minutes on a CPU, and a hyperparameter search was not affordable within that budget.

One methodological point is worth stating plainly because it is usually buried. Selection among three models on a single validation split, with 19 and 25 errors separating the top two, is not a statistically meaningful comparison. The appropriate conclusion is not that B is the better architecture but that B and C are indistinguishable and both are better than A. Reporting a winner from that comparison, without a confidence interval, would overstate what 22,908 correlated windows can support.

Finally, the defect described in Section 5.3 is part of the result rather than an embarrassment to be omitted. Three models scoring at chance on a dataset that looks perfectly learnable is a strong signal that the data, not the models, are wrong, and the fix was found by measuring fault coverage on real episodes instead of trusting a self-reported metric. That habit, more than the final accuracy, is what makes the numbers here worth anything.

SECTION 9 VIDYUT / EE14

Applications, limitations and future scope

9.1 Where the method could be used

Two applications are defensible with the evidence available. The first is offline analysis of disturbance records: a tool that labels archived fault records by category would reduce the manual work in post-event

review, and its errors would be caught by an engineer reading the same record. The second is as a benchmark component in protection research, where a fast classifier can triage simulated cases before a detailed study.

Neither application requires real-time execution, and neither is a protection function. A scheme that trips a breaker needs deterministic timing, validated behaviour on unseen topologies, and a failure mode that is safe; none of those properties were tested here.

9.2 Limitations

1. Simulated data. Every episode is an EMT simulation of one fixed topology. No claim about recorded measurements is supported.
2. Windows lie inside the fault. Mean coverage is 0.964, so the classifier is asked a question that field operation makes harder: whether a fault has started at all.
3. One seed, one split, one evaluation. No confidence intervals, no cross-validation, no variance across runs.
4. A 15.5% training subset. The forests in particular were not given the data they could use.
5. No noise, no missing channels, no sensor model. Instrument transformers, saturation, and communication loss are absent from the benchmark.
6. Balanced classes. Field fault populations are heavily skewed toward SLG; nothing here tests rare-class behaviour.
7. A and C were never evaluated on the test split. Only the selected model has a test number, which is correct protocol and a narrower result than a three-way test comparison.

9.3 Future work

The work divides naturally into three stages, and only the first is complete.

Stage 1, simulation-based classification, is this report: four classes, episode-wise split, three pipelines, one selected model, two evaluation units.

Stage 2, robustness, would add additive noise at defined signal-to-noise ratios, drop channels to emulate measurement loss, vary the sampling rate, and introduce topologies the model never saw. Each of these is a controlled perturbation of the same benchmark and would show where the decision boundary actually depends on the simulator.

Stage 3, field validation, needs recorded fault data with instrument transformers in the loop, and would have to address inference latency, calibration and the class distribution of a real network. Until that exists, the correct description of this work is a classifier that works on a benchmark.

SECTION 10 VIDYUT / EE14

Conclusion

This project built and evaluated a fault classifier for a 90 kV double-circuit transmission network using the PROTECT-90 benchmark. Episodes were split 70/15/15 before windowing, 256-sample windows were drawn from inside the fault, and 192 descriptors per window were computed in physical units. Three pipelines were trained on identical data: a random forest on descriptors, a compact one-dimensional CNN, and a hybrid of the two.

On validation the CNN reached 99.92% accuracy and 0.9992 macro-F1, 3.8 percentage points above the descriptor forest, and was selected on macro-F1 alone. On the untouched test split it reached 99.90% window accuracy with 22 errors in 22,519 windows, and 99.63% macro-F1 over 1,354 episodes under majority-vote aggregation. Its errors concentrate at the LLG boundary, where fifteen of the twenty-two mistakes originate.

The strongest defensible conclusion is comparative rather than absolute: on this benchmark, a learned temporal representation is worth about four points of accuracy over four statistics per channel, and the residual error is a specific class-boundary problem rather than a general failure of the model. The absolute accuracy is a property of a noise-free simulation with faults guaranteed to be present in the window, and it should be read as an upper bound on what the same method would achieve on recorded data.

SECTION VIDYUT / EE14

References

[1] G. Kordowich, J. Oelhaf, C. Bergler, A. Maier, S. Bayer, and J. Jäger, “PROTECT-90: A fault dataset for power system protection,” version 1.0.0, Zenodo, Jul. 2026, doi: 10.5281/zenodo.21109169.

[2] J. Oelhaf, G. Kordowich, C. Bergler, A. Maier, J. Jäger, and S. Bayer, “PROTECT-90: A fault dataset for power system protection,” arXiv:2606.24298, Jun. 2026, to appear in Proc. IEEE PES ISGT Europe, doi: 10.48550/arXiv.2606.24298.

[3] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[4] S. Kiranyaz, O. Avci, O. Abdeljaber, T. Ince, M. Gabbouj, and D. J. Inman, “1D convolutional neural networks and applications: A survey,” Mechanical Systems and Signal Processing, vol. 151, art. 107398, 2021, doi: 10.1016/j.ymssp.2020.107398.

[5] O. Turanlı and Y. Benteşen Yakut, “Classification of faults in power system transmission lines using deep learning methods with real, synthetic, and public datasets,” Applied Sciences, vol. 14, no. 20, art. 9590, 2024, doi: 10.3390/app14209590.

[6] M. Jamil, S. K. Sharma, and R. Singh, “Fault detection and classification in electrical power transmission system using artificial neural network,” SpringerPlus, vol. 4, art. 334, 2015, doi: 10.1186/s40064-015-1080-x.

[7] N. A. Tunio et al., “Fault detection and classification in overhead transmission lines through comprehensive feature extraction using temporal convolution neural network,” Engineering Reports, 2024, doi: 10.1002/eng2.12950.

[8] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2019, pp. 8024–8035.

APPENDIX VIDYUT / EE14

Appendix A Reproduction

The pipeline is reproducible from the released dataset and the project repository. Requirements are pinned in requirements.txt: numpy 2.3.5, pandas 3.1.0, scikit-learn 1.9.1, joblib 1.6.0, matplotlib 3.11.2 and torch 2.9.1, tested on CPython 3.14 on Windows x86-64 with CPU-only PyTorch.

The sequence is: place the dataset archive and metadata CSV in the project root, run preprocess.py to build the episode splits and the binary window arrays, run verify_pipeline.py to confirm split integrity and window shapes, then run train_hybrid.py to train and evaluate the three pipelines. Evaluation artefacts are written to hybrid_results/, and evaluate_episode_level.py adds the episode-level view from the saved checkpoint.

Two cautions apply. The preprocessing step rewrites roughly 6.4 GB of binary window data and takes about eight minutes; and the training subset is fixed at 16,384 windows, so a run on the full split would not reproduce these exact numbers without also changing the reported protocol.

Table 9. Files produced by the pipeline and the claim each supports.

Artefact	Contents
preprocessed/*.x.bin, *.y.bin	window arrays and labels per split
preprocessed/train_episodes.csv	episode identifiers per split
hybrid_results/exp{A,B,C}_val_metrics.json	validation metrics and matrices
hybrid_results/final_test_metrics.json	test metrics and confusion matrix
episode_evaluation/episode_level_metrics.json	episode-level results
window_signal_diagnostic.json	fault coverage before and after the fix

A.2 Known deviations from an ideal protocol

Four deviations should be declared by anyone reading these numbers as a baseline for further work. None of them invalidates the results; together they bound what the results can be compared against.

1. The training subset is 16,384 of 105,728 available windows. A model trained on the full split is not comparable to these numbers without also reporting the subset.
2. One seed, one split, one evaluation. There is no estimate of run-to-run variance, so differences smaller than a few tenths of a percentage point should not be read as effects of architecture.
3. The window-to-episode mapping is reconstructed from the split order and the windowing function, then verified against the stored arrays to within 3.8e-06. It is not read from a persisted index, because none was written.
4. Only the selected model has a test number. A and C were never scored on the test split, so no three-way comparison on held-out data exists.

The archived legacy results, produced on pre-fault windows, are retained separately and are not part of any table in this report. They are cited once, in Section 5.3, because they are the reason the sampler was rebuilt.